

Retrieval-Augmented Generation: Finding the Evidence Before Answering

Unit 5 of the Generative AI and LLM crash course

GenAI crash course

2026-10-09

Contents

Session map	2
1. Terms: the term tree	3
2. Remember	4
3. Understand: the derivation	5
3.1 Tier 1 (easy): intuition and a picture	5
3.2 Tier 2 (medium): chunking and sparse retrieval	6
3.2.1 Chunking	6
3.2.2 BM25 and TF-IDF	7
3.3 Tier 3 (medium-hard): dense retrieval, fusion and vector indexes	8
3.3.1 Dense retrieval and LSA	8
3.3.2 Combining retrievers: reciprocal rank fusion	8
3.3.3 Do the retrievers differ? An honest comparison	9
3.3.4 Vector indexes: speed against accuracy	9
3.4 Tier 4 (hard): context, grounding and when not to answer	11
3.4.1 Building the context	11
3.4.2 Grounded generation	11
3.4.3 Abstention	11
3.5 Tier 5 (hardest): open questions	12
4. Apply	12
4.1 Worked problems	12
4.2 Code and figures	12

5. Analyze	13
5.1 Limits	13
5.2 Which stage dominates the losses?	13
5.3 Sensitivity	13
5.4 Failure modes	14
5.5 Uncertainty	14
6. Evaluate	14
7. Create	15
8. Visualization lesson	15
9. Problem set	16
10. Retrieval practice and recap	16
11. Where next	16

Session map

Outcome of this session

After this session you can:

1. explain why an LLM needs retrieval (knowledge limits and hallucination) and draw the RAG pipeline: ingest, chunk, index, retrieve, build context, generate;
2. implement and compare **BM25**, **TF-IDF** and a **dense (LSA) retriever**, fuse them, and measure them with $\text{recall}@k$ and MRR on a real question set;
3. choose chunk size and overlap using an **equal-cost** comparison, not a misleading one;
4. explain how a **vector index** (IVF) trades accuracy for speed, and derive its cost;
5. say when the system should answer and when it should say “I don’t know”, and measure how well a simple signal does it.

- **Prerequisites:** Unit 3 (embeddings, cosine similarity, next-token generation), Unit 4 (context budgeting, the paired bootstrap interval, the winner’s curse). The retrievers here are built from scratch with NumPy.
- **Difficulty ladder used:** easy $\rightarrow$ medium $\rightarrow$ medium-hard $\rightarrow$ hard (Sections 3.1 to 3.4); the hardest tier is posed as open questions in Section 3.5.
- **Deliverables:** this document (.tex, .pdf, .docx), 5 figures, the module `codes/rag/rag_np/` (chunking, BM25, TF-IDF, LSA, rank fusion, an IVF index, context building, an extractive answerer, metrics; 10 tests; the course text as a corpus and 50 evaluation questions), the *Ask the course* and *Vector search* widgets, and a Colab notebook for the “RAG chatbot over your documents” practical.

- **Source anchors:** Lewis et al. 2020, arXiv:2005.11401 (RAG: combines a pre-trained generator, “parametric memory”, with a retrieval index, “non-parametric memory”; their Wikipedia index has 21M passages of 100 words, searched with a FAISS index using HNSW); Karpukhin et al. 2020, arXiv:2004.04906 (dense passage retrieval); Thakur et al. 2021, arXiv:2104.08663 (BEIR; its abstract reports that “BM25 is a robust baseline”). IDs and the quoted statements were checked on the day of writing.
- **What is measured here, and what is not.** Everything in this session is measured on **one small corpus:** the text of the first four units of this course (125{,}424 characters), with **30 answerable questions** whose answers are at known places, 10 off-topic questions and 10 on-topic questions the corpus does not answer. With 30 questions one question is 3.3 percentage points, so many differences below are **within noise**, and the text says so. There is no neural embedding model or language model on the machine that built this course: the “dense” retriever is a classical one (LSA), the “generator” is an extractive baseline that can only quote, and the practical with real models is a Colab notebook that was **not executed** while writing this document.

1. Terms: the term tree

Term	Symbol	One-line meaning	Where it appears	New / known
retrieval-augmented generation (RAG)		look up evidence first, then generate an answer from it	3.1	new
parametric vs non-parametric memory chunking		knowledge in the model’s weights / in an external index	3.1	new
BM25		cutting documents into searchable pieces word-statistics score that rewards rare matched words, with saturation	3.2	new
TF-IDF		term frequency times inverse document frequency, compared by cosine	3.2	new
LSA (latent semantic analysis)		SVD of the TF-IDF matrix: a classical dense embedding	3.3	new
dense retrieval		search by closeness of learned embedding vectors	3.3	new
hybrid search		combine several retrievers	3.3	new

Term	Symbol	One-line meaning	Where it appears	New / known
reciprocal rank fusion (RRF)		fuse rankings by summing $1/(k + \text{rank})$	3.3	new
vector database / index		stores vectors and finds near neighbours fast	3.3	new
IVF (inverted file index)		k-means cells; search only the nearest cells	3.3	new
recall@ k , MRR		share of questions answered in the top k ; mean reciprocal rank	3.2	new
groundedness		the answer is supported by the retrieved text	3.4	new
abstention		answering “I don’t know” when evidence is missing	3.4	new
embedding, cosine similarity	cos	vector for a text; angle-based closeness	3.3	known

Terms *used but not taught here*: neural embedding models and re-rankers (named, not built), approximate-search structures other than IVF such as HNSW (named in Lewis et al.), fine-tuning (Unit 6), faithfulness metrics and LLM judges (Unit 9).

2. Remember

RAG

Retrieval-augmented generation answers a question in two steps. (1) **Retrieve**: search an external collection of documents for the passages most relevant to the question. (2) **Generate**: give those passages and the question to the language model and ask it to answer *from the passages*. Lewis et al. (2020) describe it as combining a generator’s **parametric memory** (what is stored in its weights) with a **non-parametric memory** (a searchable index) that can be inspected and updated.

The pipeline

Ingest (load and clean) → **chunk** (cut into pieces) → **index** (build the search structure) → **retrieve** (top- k chunks for a query) → **build context** (fit them in the prompt budget, Unit 4) → **generate** (answer, ideally with citations) → **evaluate**.

Retrieval metrics

For Q questions, each with a gold passage: **recall@ k** is the share of questions whose gold passage is among the top k results; **MRR** (mean reciprocal rank) is $\frac{1}{Q} \sum_q 1/r_q$ with r_q the rank of the first correct result (0 if absent). In this session a result is correct when the chunk contains the gold phrase that answers the question.

Memory hooks.

- *RAG = an open-book exam.* The model does not need to remember the fact; it needs to find the page.
- *Garbage in, garbage out, twice.* A wrong passage retrieved is worse than none: the model will confidently use it.
- *The answer must survive every stage.* It can be lost in chunking, in retrieval, in the context budget, and in generation (Figure 1).

Recall quiz (answers in Section 10).

1. Why does a language model need retrieval at all? Give two reasons.
2. What does a BM25 score reward, and what does it ignore?
3. Why can a chunk boundary make a question unanswerable?
4. Why compare retrievers at equal retrieved *text*, not at equal k ?
5. What does an IVF index give up in exchange for speed?

3. Understand: the derivation

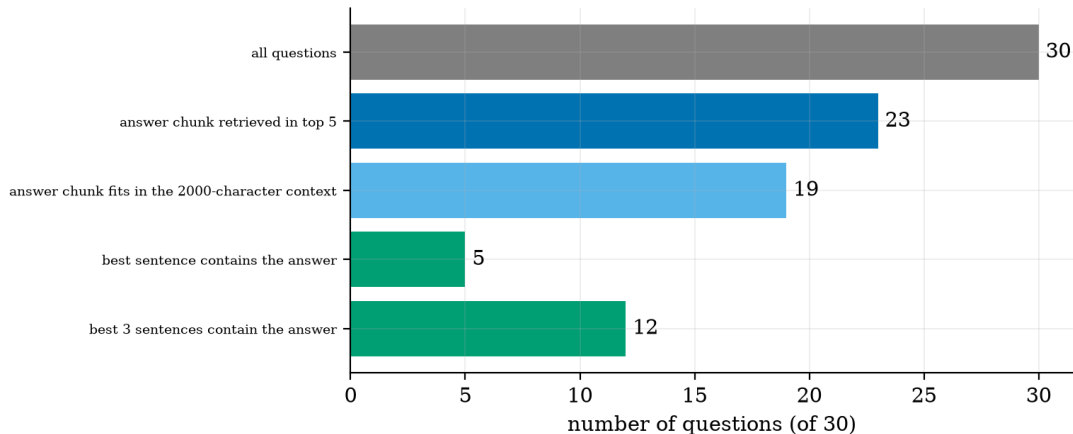
3.1 Tier 1 (easy): intuition and a picture

A language model knows what was in its training text up to a date (Unit 3) and has no way to look anything up. Ask it about your university’s rules or last week’s news and it will produce something *plausible*, which is what its training rewards (Unit 1, Section 3.4.2): this is **hallucination**. Two limits cause it: **knowledge limits** (the fact was never in the weights, or is out of date) and **no provenance** (it cannot show a source).

RAG replaces “remember it” by “look it up, then read it”. The model’s job becomes easier (read a paragraph and answer) and the knowledge becomes *editable*: change a document, not the model.

Figure 1 shows what happens to 30 real questions when I run a complete, deliberately simple pipeline over the text of this course. It is the most important picture of the session: **every stage loses questions**.

Read it from top to bottom. **7 questions are lost by retrieval** (the answer chunk is not in the top 5), **4 by the context budget** (retrieved, but the 2000-character limit leaves no room for it), and **7 more by the final step**, where the crude generator quotes the wrong sentence. The generator is far weaker than a language model, so the bottom bars say little about a real system; what carries over is the *shape*: you cannot judge a RAG system by its final answers alone, because three different things can fail.



The funnel for 30 questions about the course text, with BM25 over 600-character chunks. 23 of 30 have the answer chunk in the top 5; 19 still have it after the context is limited to 2000 characters (4 are cut by the budget); an extractive “generator” that quotes the single best sentence contains the answer for only 5, and for 12 if it quotes the best 3 sentences.

3.2 Tier 2 (medium): chunking and sparse retrieval

3.2.1 Chunking

An LLM prompt cannot hold a whole library, and a single embedding or word-count vector for a whole book would be a blur. So documents are cut into **chunks**, each searched separately. Two settings matter: the **chunk size** and the **overlap** between neighbours. The module’s `chunk_text(text, size, overlap)` cuts at about `size` characters, extends to the next space so words stay whole, and starts the next chunk `overlap` characters before the end.

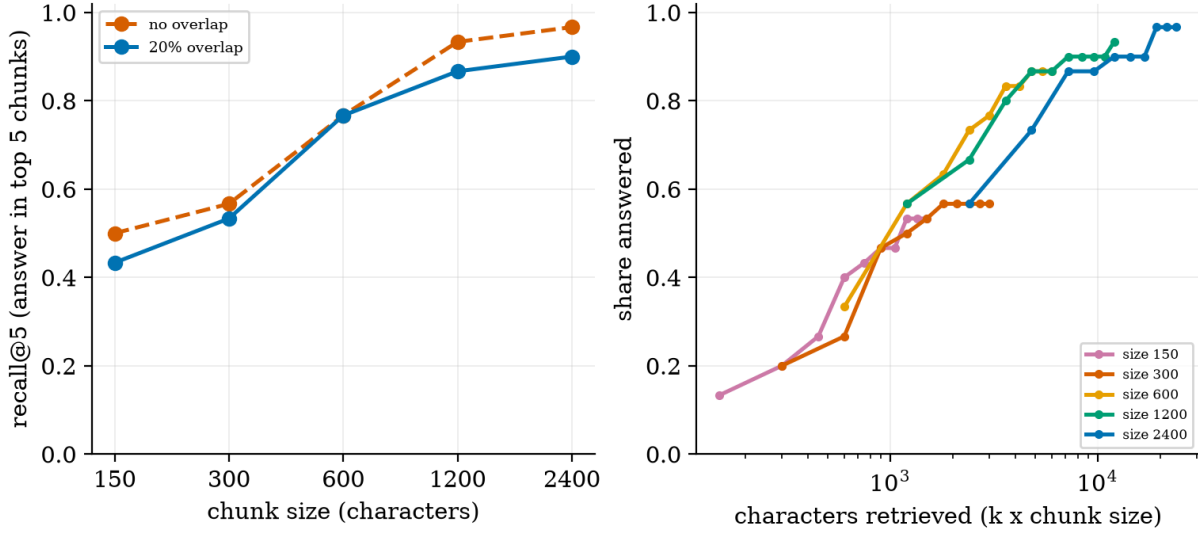
Two forces pull in opposite directions. A chunk that is too small can cut an answer in half, so that no single chunk contains it. A chunk that is too large contains the answer *and* a lot of unrelated text, which dilutes its score and wastes the context budget. Overlap prevents the first problem at the cost of storing and retrieving repeated text.

The left panel looks like “bigger chunks are better”: `recall@5` climbs from 0.50 at 150 characters to 0.97 at 2400 (no overlap). But five chunks of 2400 characters are 12{,}000 characters of context, while five chunks of 150 are only 750. **The comparison is unfair.** The right panel puts every size on the same axis, the amount of text retrieved. Now:

recall at about equal retrieved text (BM25, 20% overlap; “n/a” where no whole number of chunks falls in the range; the 1200-character row for size 150 is $k = 8$, for 300 $k = 4$, for 600 and 1200 $k = 2$ and $k = 1$). At 1200 characters all sizes are within 0.50 to 0.57, i.e. within two questions of each other, which is noise. At about 3000 characters the 600-character chunks are best of those measured, but again by a few questions.

text retrieved	chunk size 150	300	600	1200	2400
about 1200 characters	0.53	0.50	0.57	0.57	n/a
about 3000 characters	n/a	0.57	0.77	0.67 to 0.80	0.57

Two further measurements on the left panel: **answers cut by a chunk boundary** (the gold phrase appears in no chunk) number 3 at size 150 and 2 at size 300 without overlap, and 2 and 0 with 20% overlap; at size 600 and above none are cut. And overlap *lowered* `recall@5` at large sizes ($0.933 \rightarrow 0.867$



Left: recall@5 for BM25 against chunk size, without and with 20% overlap. Right: the same retrieval on an equal-cost axis: recall against the number of characters retrieved ($k \times \text{chunk size}$), for each chunk size (20% overlap).

at 1200, $0.967 \rightarrow 0.900$ at 2400): two questions each, so I do not claim a cause. One thing I did measure is that with overlap, a top-5 list more often contains two adjacent, overlapping chunks (87% of lists at size 600, against 63% without overlap), which spends slots on near-duplicates; at size 1200 the same measure is 73% against 77%, so that cannot be the whole story.

3.2.2 BM25 and TF-IDF

Both score a chunk against a query by *words*. **TF-IDF** weights a word by how often it occurs in the chunk (term frequency, damped as $1 + \ln \text{tf}$) times how rare it is across all chunks (inverse document frequency, $\ln \frac{1+N}{1+\text{df}} + 1$), turns each chunk and the query into a vector, and ranks by cosine similarity. **BM25** (Okapi BM25) is a refinement used by most search engines. For a query term t in chunk d with tf occurrences and length dl ,

$$\text{score}(q, d) = \sum_{t \in q} \text{idf}(t) \cdot \frac{\text{tf}(k_1 + 1)}{\text{tf} + k_1(1 - b + b \text{dl}/\text{avgdl})}, \quad \text{idf}(t) = \ln\left(1 + \frac{N - \text{df} + 0.5}{\text{df} + 0.5}\right), \quad (1)$$

with the usual $k_1 = 1.5$ and $b = 0.75$. Reading it term by term: **idf** rewards rare words (a word in every chunk contributes almost nothing); the fraction is **term-frequency saturation**: it grows with tf but is bounded by $k_1 + 1$, so repeating a word a hundred times does not make a chunk a hundred times better; and b **normalises length**, penalising chunks longer than average.

Worked example. Chunks *apple banana*, *apple apple cherry*, *durian*; query *apple*. Here $N = 3$, $\text{df}(\text{apple}) = 2$, so $\text{idf} = \ln(1 + \frac{3-2+0.5}{2+0.5}) = \ln 1.6 = 0.470$, and $\text{avgdl} = 2$. For chunk 0: $\text{tf} = 1$, $\text{dl} = 2$, so the bracket is $1 - 0.75 + 0.75 \cdot 1 = 1$ and the score is $0.470 \cdot \frac{1 \cdot 2.5}{1+1.5} = 0.470$. For chunk 1: $\text{tf} = 2$, $\text{dl} = 3$, the bracket is $0.25 + 0.75 \cdot 1.5 = 1.375$ and the score is $0.470 \cdot \frac{2 \cdot 2.5}{2+1.5 \cdot 1.375} = 0.470 \cdot \frac{5}{4.0625} = 0.5785$. The module returns exactly $(0.4700, 0.5785, 0)$ (test `test_bm25_matches_hand_computation`). **Saturation in numbers:** at length-bracket 1, doubling tf from 2 to 4 raises the factor from $\frac{2 \cdot 2.5}{3.5} = 1.43$ to $\frac{4 \cdot 2.5}{5.5} = 1.82$, a 27% gain rather than 100%.

Neither method understands meaning: “automobile” and “car” are different words to them.

3.3 Tier 3 (medium-hard): dense retrieval, fusion and vector indexes

3.3.1 Dense retrieval and LSA

Dense retrieval represents a text by a short vector of real numbers, an **embedding** (Unit 3), trained so that texts with related meaning point in similar directions. Search then means finding the nearest vectors to the query vector, usually by cosine similarity $\cos(u, v) = u^\top v / (\|u\| \|v\|)$. Modern systems use neural embedding models for this (for example the encoders of Karpukhin et al. 2020, trained so that a question is close to its answer passage); the Colab notebook of this unit uses one. I cannot run such a model here, so I use the classical ancestor, **latent semantic analysis (LSA)**.

LSA takes the TF-IDF matrix $X \in \mathbb{R}^{N \times V}$ (one row per chunk) and computes its singular value decomposition $X = U \Sigma V^\top$. Keeping only the k largest singular values gives the best rank- k approximation of X (the classical Eckart-Young theorem, quoted, not re-derived). Chunk i becomes the k -vector $d_i = (XV_k)_i$, and a query vector q is mapped by $V_k^\top q$. Why this helps: words that appear in the same kinds of chunks (car and automobile both occur with engine, repair, tyre) end up with similar columns, so they collapse onto the same latent direction and a query can match a chunk that does not contain its exact words.

A six-chunk toy shows it. Chunks: car engine repair, automobile engine repair, banana smoothie recipe, car tyre repair, automobile tyre service, mango banana smoothie. Query automobile repair:

TF-IDF sees **car tyre repair** as only sharing the word *repair* with the query (0.331). LSA places it level with the exact-word chunk (0.947), because *car* and *automobile* co-occur with the same words in this tiny collection. This is a six-sentence toy built to show the mechanism; on real text the effect is much noisier.

chunk	TF-IDF cosine	LSA ($k = 3$) cosine
car engine repair	0.331	0.684
automobile engine repair	0.795	0.947
banana smoothie recipe	0.000	0.000
car tyre repair	0.331	0.947
automobile tyre service	0.409	0.710
mango banana smoothie	0.000	0.000

3.3.2 Combining retrievers: reciprocal rank fusion

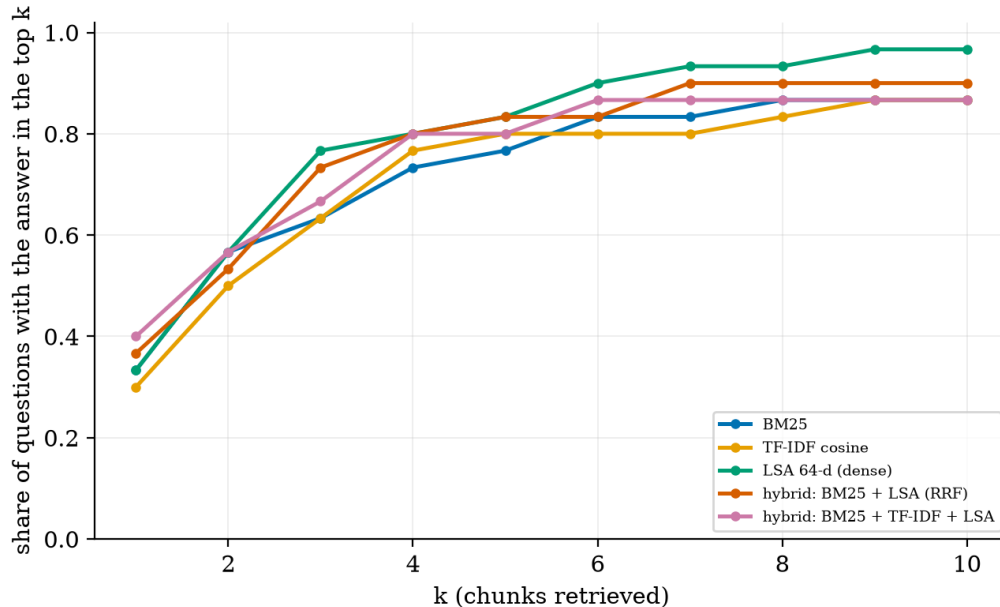
BM25 scores are unbounded, cosine scores lie in $[-1, 1]$; adding them needs a careful rescaling. **Reciprocal rank fusion** avoids scores and uses only ranks: for each retriever r and chunk d at position $\text{rank}_r(d)$ (1 for the best),

$$\text{RRF}(d) = \sum_r \frac{1}{c + \text{rank}_r(d)}, \quad c = 60. \quad (2)$$

Worked example, two retrievers over three chunks: one ranks them (0, 1, 2), the other (2, 1, 0). Chunk 1 is second in both: $2/(60 + 2) = 0.032258$. Chunk 0 is first in one and third in the other: $1/61 + 1/63 = 0.032266$, and so is chunk 2. The fused order is therefore (0, 2, 1): **a chunk that one retriever loves beats a chunk that both find only mediocre**, by a hair, which is the behaviour you want from a union of different signals. (The constant $c = 60$ is the commonly used default; I did not tune it.)

3.3.3 Do the retrievers differ? An honest comparison

Figure 3 and the table below give recall for the four retrievers (600-character chunks with 120 overlap, 257 chunks, the same 30 questions).



Share of the 30 questions with the answer chunk in the top k , for BM25, TF-IDF cosine, an LSA 64-dimensional dense retriever and two rank-fusion hybrids.

retrieval quality. Differences of one or two questions are not evidence. The paired comparison of Unit 4 settles it for recall@5: **LSA minus BM25 is +0.067 with a 95% bootstrap interval of $[-0.067, +0.200]$** (LSA finds 3 questions BM25 misses, BM25 finds 1 that LSA misses); the hybrid minus BM25 is +0.067 with interval $[0.000, +0.167]$; TF-IDF minus BM25 is +0.033 with $[0.000, +0.100]$. **No difference is established with 30 questions.** This agrees in spirit with the BEIR benchmark (Thakur et al. 2021), which reports that BM25 is a robust baseline: a plain word-matching search is hard to beat on many collections. What does hold up is a *pattern*: the questions BM25 misses and LSA catches (3 of them) are ones where the question and the passage use different words for the same idea, which is exactly what a dense retriever is for.

method	recall@1	@3	@5	@10	MRR
BM25	0.333	0.633	0.767	0.867	0.528
TF-IDF cosine	0.300	0.633	0.800	0.867	0.500
LSA 64-d (dense)	0.333	0.767	0.833	0.967	0.552
hybrid BM25 + LSA (RRF)	0.367	0.733	0.833	0.900	0.557
hybrid BM25 + TF-IDF + LSA	0.400	0.667	0.800	0.867	0.571

3.3.4 Vector indexes: speed against accuracy

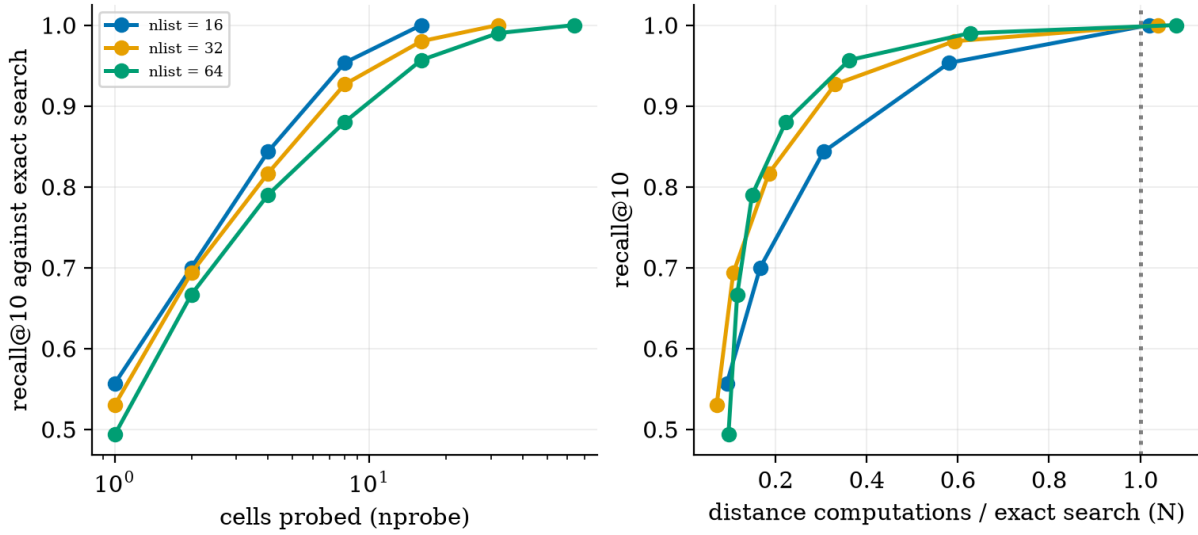
Exact nearest-neighbour search compares the query with every stored vector: N distance computations. For millions of chunks that is too slow, so vector databases use **approximate nearest-neighbour** structures. The **inverted file (IVF)** index runs k -means once to split the vectors into `nlist` cells, each with a centre. A query is compared with the `nlist` centres, and then only with the vectors in the

$nprobe$ closest cells. The cost per query, with cells of about equal size $N/nlist$, is

$$\text{cost}(nlist, nprobe) \approx nlist + nprobe \cdot \frac{N}{nlist}. \quad (3)$$

Minimising over $nlist$ (set the derivative $1 - nprobe N/nlist^2$ to zero) gives $nlist^* = \sqrt{nprobe N}$ and a minimum cost of $2\sqrt{nprobe N}$. Worked checks: for $N = 806$ and $nprobe = 8$, $nlist^* = 80$ and the cost is 161 computations, 20% of N ; for $N = 10^8$ and $nprobe = 16$ the cost is 80,000, which is 1250 times fewer than exact search.

The price is that the true neighbour may sit in a cell that is not probed (usually near a border), so recall falls below 1. I measured it on 806 real LSA chunk vectors (150-character chunks, 64 dimensions) with the 30 question vectors as queries, comparing with exact search for the top 10:



Recall@10 of an IVF index against exact search on 806 real chunk vectors, as the number of probed cells grows (left), and the same recall against the work done as a fraction of an exact search (right), for 16, 32 and 64 cells.

with 64 cells and 8 probed, recall is 0.88 at 22% of the work, close to the 20% predicted by (3). Probing every cell is exact but costs *more* than exact search (1.02 to $1.08 \times N$), because of the centre comparisons. **At 806 vectors an index is not worth it**; the saving appears only for large collections, which is why small projects can search exactly.

cells searched (nprobe)	1	2	4	8	16
$nlist = 16$: recall / work ($\times N$)	0.56 / 0.09	0.70 / 0.17	0.84 / 0.31	0.95 / 0.58	1.00 / 1.02
$nlist = 64$: recall / work ($\times N$)	0.49 / 0.10	0.67 / 0.12	0.79 / 0.15	0.88 / 0.22	0.96 / 0.36

3.4 Tier 4 (hard): context, grounding and when not to answer

3.4.1 Building the context

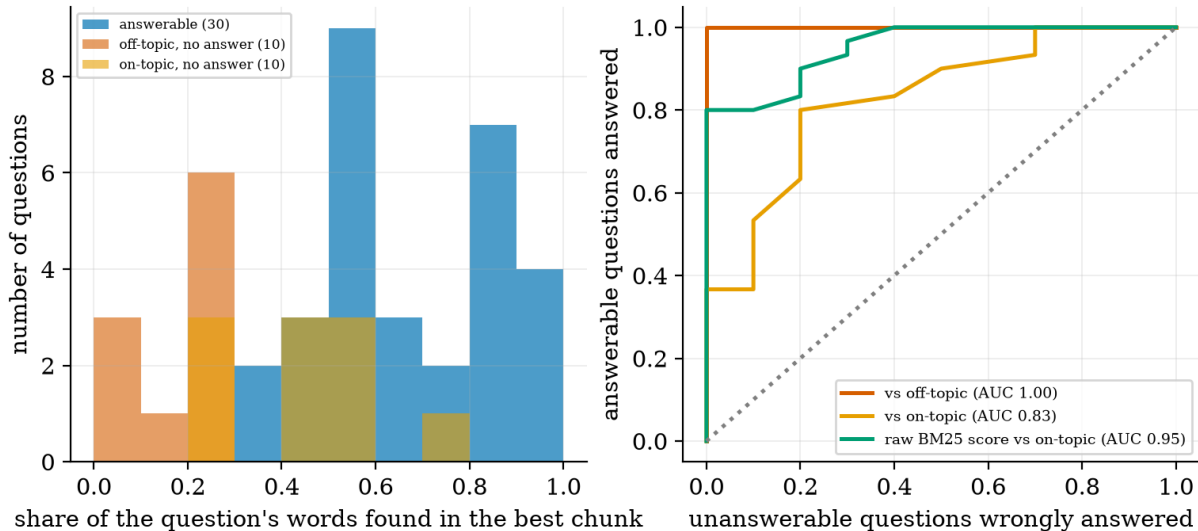
After retrieval, the chunks must be placed in the prompt (Unit 4). The module's `build_context` takes chunks in rank order until a character budget is used. Three practical rules follow from earlier sessions: respect the budget (Figure 1: 4 questions were lost here); put the strongest evidence at the start or end of the context, because models use the middle of a long context worst (Liu et al., Unit 4); and remove near-duplicates, which overlapping chunks create.

3.4.2 Grounded generation

The prompt tells the model to answer *only* from the passages, to cite which passage it used, and to say it does not know if they do not contain the answer. An answer is **grounded** if every claim is supported by the retrieved text. The module's extractive answerer is grounded by construction, because it can only quote. A language model is not: it can still add claims from its own memory, so groundedness has to be checked (Unit 9).

3.4.3 Abstention

A RAG system must sometimes say “I don’t know”. A simple signal is the **coverage** of the best chunk: the share of the question’s content words that occur in it. I tested it on two kinds of unanswerable questions: 10 **off-topic** ones (“What is the capital of Australia?”) and 10 **on-topic near-misses** whose key term is absent from the corpus (“How does LoRA reduce the number of trainable parameters?”, “What dataset was BERT pre-trained on?”; I checked that the key term does not occur anywhere in the corpus).



Left: coverage of the best chunk for answerable questions (blue), off-topic unanswerable (red) and on-topic unanswerable (orange). Right: for every threshold, the share of answerable questions answered against the share of unanswerable ones wrongly answered, for coverage against off-topic (red) and on-topic questions (orange) and for the raw BM25 score against on-topic questions (green).

Mean coverage is 0.675 for answerable, 0.152 for off-topic and 0.436 for near-miss questions. Against

off-topic questions the signal separates perfectly (AUC 1.0, no overlap at all), which is *too easy* and would have given a false impression of success. Against the near-misses, which share vocabulary with the corpus, the area under the curve is 0.83: choosing a threshold of 0.5 answers 83% of the answerable questions but also **wrongly answers 40% of the near-misses**; at 0.6 the figures are 63% and 20%; at 0.7, 43% and 10%. There is no free lunch: refusing more of the unanswerable also refuses answerable ones. In this small test the *raw BM25 score* did better than coverage on the near-misses (AUC 0.95), a reminder to compare simple signals rather than assume. With 10 near-miss questions both AUC values are uncertain.

3.5 Tier 5 (hardest): open questions

1. **When does dense retrieval beat BM25?** BEIR found BM25 a strong baseline across many collections; neural retrievers shine when queries and passages use different words and when they are trained on similar data. Which wins on *your* collection can only be measured on your questions, and with far more than 30.
2. **How should documents be chunked?** Fixed-size chunks are crude: they ignore headings, tables and sentence boundaries. Structure-aware chunking is better in principle, but there is no agreed way to compare strategies except end-to-end evaluation.
3. **Can the generator be trusted to use the evidence?** A model may ignore the passages, or follow a wrong passage. Measuring faithfulness of the answer to the context (not just correctness) is unsettled (Unit 9).
4. **Multi-step questions.** If the answer needs facts from two documents, one retrieval step is not enough; repeated retrieval inside an agent loop is the subject of Unit 8.

4. Apply

4.1 Worked problems

Problem 1 (BM25 by hand). Chunks `apple banana, apple apple cherry, durian`, query `apple`: scores (0.470, 0.5785, 0), computed in Section 3.2.2.

Problem 2 (rank fusion). Rankings (0, 1, 2) and (2, 1, 0), $c = 60$: RRF values (0.032266, 0.032258, 0.032266); fused order (0, 2, 1).

Problem 3 (recall and its uncertainty). With 30 questions, $\text{recall@5} = 0.767$ means 23 hits. A 95% normal interval is $0.767 \pm 1.96\sqrt{0.767 \cdot 0.233/30} = 0.767 \pm 0.151$, i.e. [0.62, 0.92]. One more or fewer hit moves recall by 0.033.

Problem 4 (IVF cost). $N = 806$, `nlist` = 64, `nprobe` = 8: (3) gives $64 + 8 \cdot 806/64 = 164.8$ computations, $0.204 N$; the measured value is $0.223 N$ (cells are not of equal size).

Problem 5 (context budget). A 2000-character budget and chunks of about 720 characters: three chunks fit, so a gold chunk ranked 4th or 5th is retrieved but never reaches the model. This is exactly the “cut from context” loss of Figure 1.

4.2 Code and figures

Everything is in `codes/rag/rag_np/` (see its README); the course text and the 50 questions are in `data/`. The heart of the retrieval pipeline:

```

corpus = R.load_corpus()
chunks = R.chunk_corpus(corpus, size=600, overlap=120)
bm25 = R.BM25([c["text"] for c in chunks])
order = R.rank(bm25.scores("What is the optimal GAN discriminator?"))
context = R.build_context(chunks, order, budget_chars=2000)
answer = R.extractive_answer(question, [chunks[i]["text"] for i in context], n=3)

```

and the evaluation: `R.evaluate(bm25.scores, chunks, queries, ks=(1, 5, 10))` returns $\text{recall}@k$ and MRR. **To build your own RAG over your documents:** replace `data/` with your text files (names `u*.txt`) and write your own questions with a gold phrase each; the tests check that every gold phrase occurs exactly once.

For real embeddings and a real language model, open the Colab notebook `u5_rag_real_models.ipynb`. It loads a small sentence-embedding model, builds the same chunk index, retrieves, and hands the evidence to a small chat model. It was **not executed** while writing this document, and the two models it names were confirmed to exist on the Hugging Face hub.

Try it in the playground

Ask the course runs this very pipeline (the same chunker, BM25 and TF-IDF, ported to JavaScript and checked to give identical rankings) over the text of Units 1 to 4 in your browser; try a question the course does not answer. **Vector search** shows an IVF index on 2-D points so you can see why neighbours near a cell border are missed.

5. Analyze

Checklist items used: limits (5.1), dominance (5.2), sensitivity (5.3), failure modes (5.4), uncertainty (5.5).

5.1 Limits

- $k = 1$: only the single best chunk is used: $\text{recall}@1$ is 0.30 to 0.40 for every retriever here.
- Chunk size $\rightarrow$ whole document: retrieval becomes document-level, $\text{recall}@5$ is high but the context is huge (Figure 2, right panel).
- `nprobe = nlist` in IVF: exact search, no saving (Figure 4).
- Query with no term in any chunk: all BM25 scores are 0 and the ranking is arbitrary (index order); coverage = 0 flags it.

5.2 Which stage dominates the losses?

In Figure 1, the three losses are comparable in size: 7 (retrieval), 4 (context budget), 7 (generation) out of 18 failures. There is no single bottleneck; improving one stage moves the problem to the next.

5.3 Sensitivity

Retrieval quality is most sensitive to *chunking*, because it changes what the retriever sees at all: $\text{recall}@5$ moves from 0.50 to 0.97 with size (but see the equal-cost caveat), while swapping BM25 for

LSA changed it by at most 0.067 (not distinguishable from noise).

5.4 Failure modes

- **Answer cut by a boundary** (2 to 3 of 30 questions at 150 to 300 characters without overlap).
- **Wrong chunk ranked high** because it repeats the question’s words (the best “answer” sentence in *Ask the course* is often a *quiz question* that repeats your words).
- **Answer retrieved but not used** (context budget, ordering, generator).
- **Answering when the evidence is missing** (40% of near-miss questions at a coverage threshold of 0.5).
- **Stale index**: documents change but the index is not rebuilt.

5.5 Uncertainty

All results rest on 30 questions drawn by one author from one corpus. The 95% interval for a recall of 0.77 is about ± 0.15 ; the retriever comparison and the overlap result are inside that noise. The corpus is also the text of *this course*, which contains many repeated terms (the quiz questions repeat the explanations), so retrieval here is easier than on a heterogeneous collection.

6. Evaluate

RAG versus the alternatives. *Prompting alone* (Unit 4) works when the facts fit in the prompt. *Fine-tuning* (Unit 6) changes behaviour and style but is a poor way to add facts that change. RAG is the usual first choice for **fresh, private or large knowledge** and gives provenance; its costs are an index to maintain, a retrieval step that can fail, and longer prompts. Lewis et al. motivate it exactly by the ability to revise knowledge and to give provenance.

Sparse versus dense versus hybrid. Sparse (BM25) is cheap, exact on rare terms and names, and needs no model; dense handles paraphrase; hybrids hedge. The measured evidence here cannot rank them (Section 3.3.3), and BEIR suggests BM25 is a hard baseline to beat, so *always run BM25 as the baseline*.

Indexes. Exact search up to a few hundred thousand vectors; approximate structures beyond. Choose n_{list} near $\sqrt{n_{\text{probe}} N}$ and measure recall against exact search on your own data.

When not to use RAG. When the answer needs reasoning over the *whole* corpus (summarise everything), when the questions are about the model’s own behaviour, or when retrieval quality cannot be checked.

Wider context. RAG moves part of “intelligence” from the model to the *information system* around it: search quality, document hygiene and access control become as important as the model.

7. Create

What you can build from this

1. **College information chatbot:** put 20 PDF-to-text files in `data/`, write 30 questions with gold phrases, and report `recall@5` with an interval.
2. **Chunking study:** repeat Figure 2 on your documents with an equal-cost axis, and add a sentence-aware chunker.
3. **Hybrid retriever:** tune the RRF constant c and test whether the hybrid beats BM25 with a paired interval.
4. **Abstention calibration:** choose a coverage threshold that answers at least 80% of answerable questions, and report the false-answer rate on near-miss questions.
5. **Combine with Unit 4:** feed `build_context` into `few_shot_messages`, pin the instructions, and use the paired comparison to judge two prompts for the answer step.
6. **Module for the open-source package:** add a BM25F (field-weighted) variant or a simple cross-encoder-style re-ranker interface, with tests.

8. Visualization lesson

charts taught here. How to read the ROC-style panel of Figure 5: the closer a curve is to the top-left corner, the better the score separates the groups; the diagonal is chance. Always say *which* negatives were used.

Chart type	Best for	How it misleads	matplotlib call
funnel (horizontal bars)	where items are lost in a pipeline (Fig. 1)	stages that are not nested break the “funnel” reading	<code>ax.barh(y, counts)</code>
equal-cost comparison (metric against resource) recall@ k curves	fair comparison of settings with different costs (Fig. 2 right) comparing retrievers across k (Fig. 3)	hides the settings’ individual curves if lines overlap curves that cross; differences of 1 to 2 questions look like gaps	<code>ax.plot(k*size, recall)</code> <code>ax.plot(ks, recall, "o-")</code>
recall-versus-work trade-off	speed-accuracy trade-offs (Fig. 4)	a log axis can make small costs look large	<code>ax.plot(cost, recall)</code>
overlapping histograms and ROC curve	a score that separates two groups (Fig. 5)	a perfect AUC on an easy negative class (off-topic)	<code>ax.hist(..., alpha =.6)</code> and <code>ax.plot(fpr, tpr)</code>

9. Problem set

Graded exercises

1. (easy) Name the stage of the RAG pipeline where each of these failures happens: (a) the answer is split between two chunks; (b) the retrieved chunk does not fit the prompt; (c) the model ignores the evidence.
2. (medium) Compute the BM25 score of the chunk `apple apple cherry` for the query `apple cherry`, using $N = 3$ chunks (`apple banana`, `apple apple cherry`, `durian`), $k_1 = 1.5$, $b = 0.75$.
3. (medium-hard) Two retrievers rank five chunks as $(3, 1, 0, 2, 4)$ and $(1, 3, 4, 0, 2)$. Compute the RRF scores with $c = 60$ and give the fused order.
4. (hard) Derive (3) for equal cells and show that the optimal number of cells is $\sqrt{\text{nprobe } N}$. For $N = 10^7$ and $\text{nprobe} = 10$, find the cost and the speed-up over exact search.
5. (hardest) A system answers whenever coverage $\geq \tau$. Out of 30 answerable and 10 unanswerable questions, 25 and 4 respectively have coverage $\geq \tau$. Compute the share of answerable questions answered and the *precision* of answering (share of given answers that were to answerable questions). Then explain why the precision changes if the mix of questions in production is 50% unanswerable.

10. Retrieval practice and recap

Answers to the Section 2 quiz.

1. Knowledge limits (facts not in, or out of date in, the weights) and no provenance (it cannot cite a source), which together produce hallucination.
2. It rewards query words that are rare across chunks and occur in the chunk (with saturation and length normalisation); it ignores meaning, word order and synonyms.
3. If the gold phrase is split across the boundary, no single chunk contains it (2 to 3 of 30 questions at 150 to 300 characters without overlap).
4. Because bigger chunks bring more text; recall per character retrieved is the fair measure (Figure 2, right panel).
5. Exactness: the true nearest neighbour can sit in a cell that is not searched, so recall is below 1.

Explain it to a friend.

- Why is RAG like an open-book exam, and what can still go wrong?
- Why can the same chatbot answer correctly about a document it has never seen?
- Why is “it answered” not evidence that “it found the right passage”?

Re-derive tomorrow without looking: (a) the BM25 score for the apple example; (b) RRF for two rankings; (c) the optimal $\text{nlist} = \sqrt{\text{nprobe } N}$.

11. Where next

- **Next:** Unit 6 (fine-tuning and adaptation): when RAG is not enough (style, format, skills), and how LoRA, QLoRA and quantization make adaptation cheap.

- **Terms met but not taught:** neural re-rankers, query rewriting, HNSW, product quantization, hybrid scoring weights.
- **Open threads:** whether dense retrieval beats BM25 on this corpus with a larger question set (Section 3.5); whether overlap hurts or only appeared to (Section 3.2.1).